AI TUTOR FOR HANDWRITTEN ASSIGNMENTS

An Automated Handwritten Assignment Evaluation System using Computer Vision, OCR and Large Language Models

Research & Project Proposal Document

Submitted for: Software Group Project

Prepared by:

Ohm Shah (D25DCS160)
Dev Shah (D25DCS172)
Kangan Pathak (D25DIT092)
Department of Computer Science and Engineering, DEPSTAR, CHARUSAT

August 2026

Table of Contents

1. Abstract 1
2. Introduction and Problem Statement 1
3. Objectives 1
4. Proposed System Overview 1
5. Module 1 — Image Preprocessing with OpenCV 3
    5.1 Preprocessing Steps 3
6. Module 2 — Optical Character Recognition (OCR) 3
    6.1 Comparison of OCR Options 3
7. Module 3 — Answer Evaluation with a Large Language Model 4
    7.1 Example Output 4
8. Module 4 — Data Storage (MongoDB) 4
9. Module 5 — Teacher and Student Dashboards 4
    9.1 Teacher Dashboard 4
    9.2 Student Dashboard 4
10. Where Is Machine Learning Actually Used? 5
11. Recommended Technology Stack 5
12. Significance of the Project 5
13. Conclusion 5
14. References 6

1. Abstract

Manually grading handwritten assignments is slow and inconsistent: a teacher with a large class may spend eight to ten hours reading, interpreting handwriting, judging correctness, and writing feedback for a single assignment. This document presents the research and design for an AI-powered assignment evaluation platform that automates this workflow end to end. The system accepts a photograph of a handwritten answer, cleans it using classical computer vision techniques, converts the handwriting to machine-readable text using deep-learning-based Optical Character Recognition (OCR), and then uses a Large Language Model (LLM) to semantically compare the student's answer against a model answer, producing marks and constructive feedback within seconds. The complete platform is built on the MERN stack (MongoDB, Express.js, React.js, Node.js) and exposes separate dashboards for teachers and students.

2. Introduction and Problem Statement

In a typical classroom of about 100 students, a teacher must read every submitted answer sheet, decipher individual handwriting styles, judge whether the answer is conceptually correct, assign marks, and write feedback for each student. This process is repetitive, time-consuming, and prone to inconsistency between different graders or even the same grader across a long session.

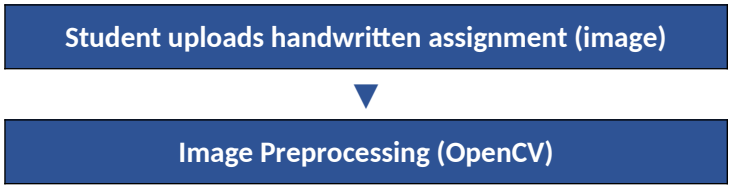
The goal of this project is to replace this manual process with an AI Tutor that can perform the same evaluation automatically: a student uploads a photograph of their handwritten answer, and within seconds the system returns marks and detailed feedback. This reduces teacher workload significantly while giving students faster, more consistent feedback on their work.

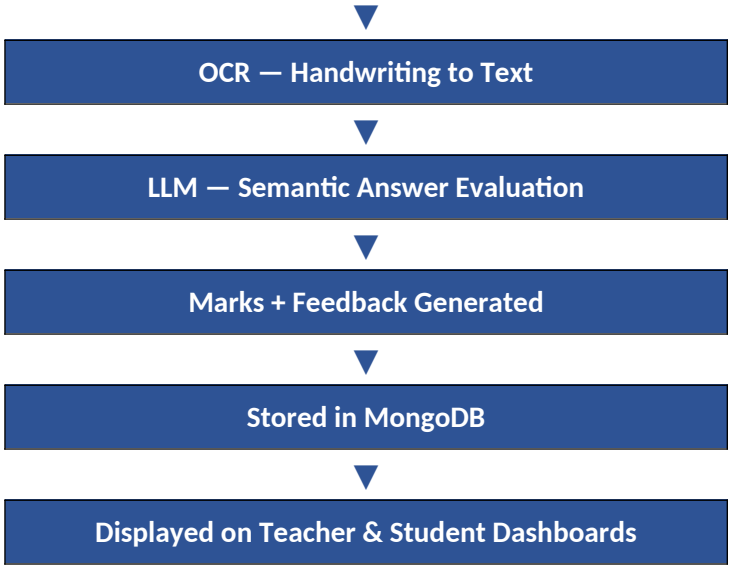
3. Objectives

- Automate the evaluation of handwritten assignments from image capture to final feedback.
- Apply classical computer vision techniques to clean noisy, tilted, or poorly lit handwritten images before recognition.
- Use a deep-learning OCR model to accurately convert handwriting into digital text.
- Use a Large Language Model to semantically evaluate the extracted answer against a teacher-provided model answer, rather than relying on rigid keyword matching.
- Generate marks and human-readable feedback automatically and store them for later review.
- Provide separate, role-based dashboards for teachers and students built on the MERN stack.

4. Proposed System Overview

The system follows a linear pipeline: a student uploads an image of their handwritten answer, which is progressively cleaned, digitised, evaluated, and stored before being surfaced back to both the student and the teacher through their respective dashboards.




OCR — Handwriting to Text
LLM — Semantic Answer Evaluation
Marks + Feedback Generated
Stored in MongoDB
Displayed on Teacher & Student Dashboards

5. Module 1 — Image Preprocessing with OpenCV

A photograph taken by a student's phone is rarely clean: it may suffer from poor lighting, shadows, tilt, blur, or background noise. OCR accuracy depends heavily on image quality, so the first stage of the pipeline uses OpenCV — a computer vision library, not a machine learning model — to prepare the image.

5.1 Preprocessing Steps

- Grayscale conversion — OCR performs better on grayscale images than on raw colour images.
- Noise removal — eliminates small artefacts (specks, dust, stray marks) that could be misread as characters.
- Thresholding (binarisation) — converts the image into pure black text on a white background, sharpening the boundary between ink and paper.
- Deskewing — straightens a photo taken at an angle so text lines are horizontal.
- Cropping — removes irrelevant background such as the desk, hands, or other objects, isolating just the answer sheet.
- Line segmentation — separates the answer into individual lines to make downstream OCR more reliable.

It is worth clarifying that OpenCV itself is not machine learning; it applies classical image-processing algorithms such as edge detection, filtering, thresholding, and contour detection.

6. Module 2 — Optical Character Recognition (OCR)

Once the image is cleaned, OCR converts the handwritten content into machine-readable text. Unlike OpenCV, modern OCR engines are themselves deep-learning models, trained on large datasets of handwritten samples, and therefore qualify as AI.

6.1 Comparison of OCR Options

OCR Engine	Strengths	Limitations	Suitability
EasyOCR	Simple to set up; fast	Comparatively weak on handwriting	Good for prototyping only
PaddleOCR	Strong handwriting recognition; fast; open source	Requires some tuning for best results	Recommended for this project
TrOCR (Microsoft)	Transformer-based; very high accuracy on handwriting	Heavier, slower inference	Best if accuracy is the top priority
Google Vision OCR	Very accurate, cloud-managed	Paid beyond free tier; external dependency	Viable but has a recurring cost

For a college-level project, PaddleOCR offers the best balance of accuracy, speed, and ease of integration. TrOCR can additionally be benchmarked against PaddleOCR to demonstrate a deeper comparative evaluation of OCR approaches within the project report.

7. Module 3 — Answer Evaluation with a Large Language Model

The OCR stage produces raw text, but the system still needs to judge whether that text represents a correct answer. This is the role of the LLM (for example GPT, Llama, Gemma, or Mistral). The teacher first supplies a model answer or rubric — for instance, for the question "Explain Photosynthesis," the key points might be that it uses sunlight, uses chlorophyll, produces glucose, and produces oxygen.

The LLM then compares the student's extracted answer against this model answer, checking relevance, presence of key concepts, missing points, and overall correctness of explanation. Crucially, this comparison is semantic rather than a simple keyword match: a phrase like "plants make food using sunlight" and "sunlight helps plants produce glucose" express closely related ideas even though the wording differs, and the LLM is able to recognise this equivalence.

7.1 Example Output

Field	Example Value
Marks	7 / 10
Reasoning	Sunlight and food production mentioned; chlorophyll and oxygen missing
Feedback	Good answer overall — mention chlorophyll and oxygen production, and explain how glucose is formed

8. Module 4 — Data Storage (MongoDB)

Every evaluation — the student's identity, the question, the assigned marks, the generated feedback, and the submission date — is persisted in MongoDB so that it can be retrieved later by both students and teachers, and so that class-level performance can be tracked over time.

9. Module 5 — Teacher and Student Dashboards

9.1 Teacher Dashboard

- Create assignments and add questions
- Upload model answers or rubrics
- View student submissions
- Review AI-generated marks and feedback, and override them where necessary
- Track class performance over time

9.2 Student Dashboard

- Upload assignment photographs
- View marks and feedback
- Review previous submissions
- Track personal improvement over time

10. Where Is Machine Learning Actually Used?

Not every stage of the pipeline is AI-driven; it is useful to be explicit about this distinction, both for architectural clarity and for the project viva.

Pipeline Stage	Technology	Is it AI?
Image cleaning	OpenCV	No — classical computer vision
Handwriting recognition (OCR)	PaddleOCR / TrOCR	Yes — deep learning
Answer evaluation	GPT / Llama / Gemma	Yes — deep learning (Transformers)
Plagiarism detection (optional extension)	Embeddings	Yes — machine learning
Diagram checking (optional extension)	CNN	Yes — deep learning

11. Recommended Technology Stack

Layer	Technology
Frontend	React.js with Tailwind CSS or Bootstrap
Backend	Node.js with Express.js
Database	MongoDB
File storage	Cloudinary or AWS S3
Image preprocessing	OpenCV
OCR	PaddleOCR (primary), optionally benchmarked against TrOCR
Answer evaluation	GPT / Llama / Gemma / Mistral (LLM API)

12. Significance of the Project

This project goes beyond a standard CRUD web application by combining several modern technology layers into a single, coherent system: the MERN stack for the web application itself, OpenCV for image preprocessing, deep-learning OCR for handwriting recognition, and an LLM for semantic answer evaluation and feedback generation, with MongoDB and cloud storage handling persistence. Rather than simply digitising the submission process, the system addresses a genuine, recurring problem — the time cost of manual grading — while giving students faster and more consistent feedback.

13. Conclusion

The AI Tutor for Handwritten Assignments demonstrates how classical computer vision, deep-learning OCR, and Large Language Models can be combined within a conventional MERN web application to automate a task that is traditionally slow and manual. The proposed architecture is modular — image preprocessing, OCR, and LLM evaluation are each independent stages — which makes the system straightforward to build incrementally, test stage by stage, and extend later with

optional features such as plagiarism detection or diagram checking. This makes it a strong candidate for a Software Group Project, since it offers clear scope for individual module ownership, measurable milestones, and a demonstrable end-to-end system by completion.

14. References

- OpenCV documentation — opencv.org
- PaddleOCR — PaddlePaddle handwriting/text recognition toolkit
- TrOCR: Transformer-based Optical Character Recognition, Microsoft Research
- Google Cloud Vision API documentation
- MongoDB, Express.js, React.js, Node.js official documentation (MERN stack)